Universidade Leiria • Oeste
Escola de Tecnologia e Gestão

Information Systems Integration

Part 4 – Service Oriented Architecture
Web Services and RESTful

Toward a Service Oriented Architecture

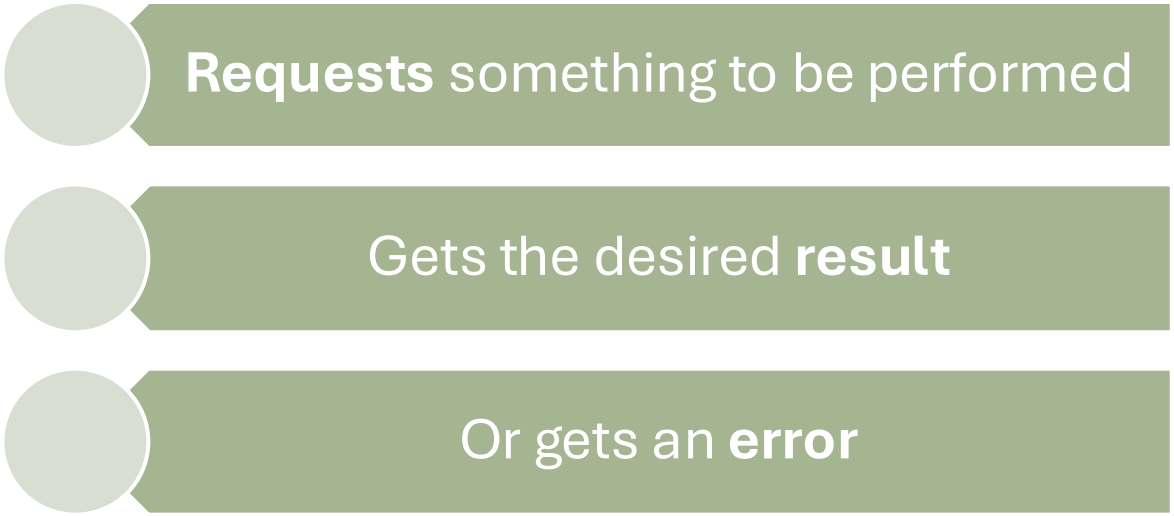
- Technologies leading to SOA (beginning…)
  - Remote Procedure Calls (RPC)
  - CORBA, Jini, DCOM, ...
- Concepts
  - Abstraction – business tasks
  - Standards

Services

- In the software world a “service”
  - Takes some **input**, performs some **work**, produces some **output**
  - **Request-response** model: client requests, server responses
- A “web service” is
  - Software service that communicates over standard Web protocols
  - Classical (heavyweight) services use SOAP, WSDL, XML, WS-*
  - Lightweight (RESTful) services use HTTP, REST and JSON

Services

- A "**client**" (consumer) uses the services



Service-Oriented Architecture (SOA)

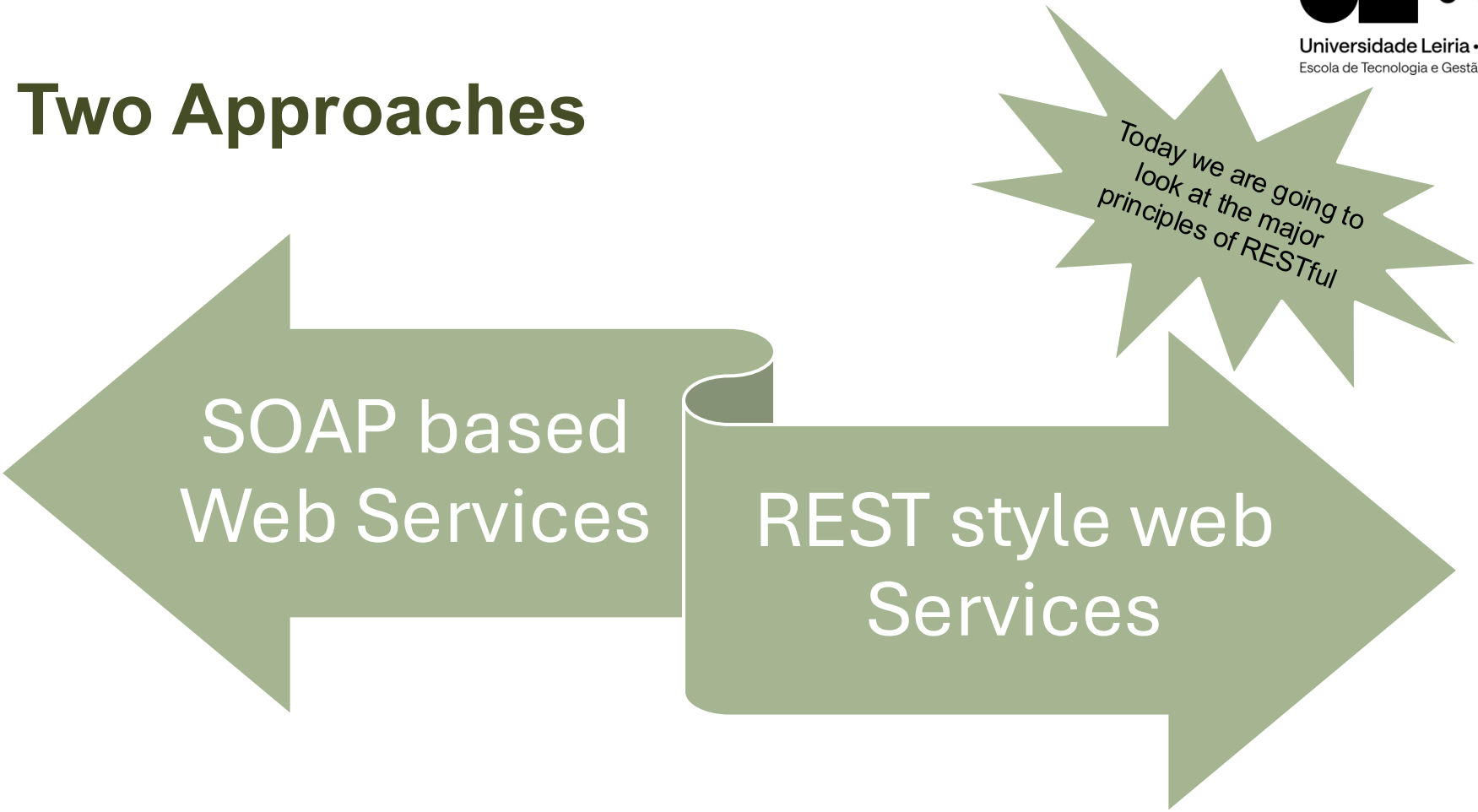
- SOA is an architectural **concept** for development of software systems
  - Using reusable building blocks called “service”
  - SOA = decouple the monolithic software to reusable services
- Service in SOA are:
  - Autonomous, **stateless** business functions
  - Accept **requests** and return **responses**
  - Use well-defined, standard interface (standard protocols) – HTTP, JSON, XML, SOAP, etc…

What are Web Services?

- The idea:
  - "to provide the technologies to allow software in different places, written in different languages and resident on different platforms to connect and interoperate."

From "Programming the World Wide" by Sebesta

Two Approaches



Background

- The Web technologies
  - 1.Designed to deliver web pages between HTTP servers and clients **(WEB=HTTP+URIs+HTML)**
  - 2.Then, information delivery and service platform
  - 3.Later (90'), Web foundations were used to support RPC
    - Simple Object Access Protocol (SOAP)
    - Representational State Transfer (RESTful)

So, why RESTul?
(Isn't SOAP the right option?)

Universidade Leiria • Oeste
Escola de Tecnologia e Gestão

Representational State Transfer (REST)

RESTful Web Services

REST - Representational State Transfer

The REST viewpoint [2]

- **REST sees the Web as an incredibly scalable and well-designed information system.**
- REST relies on:
  - **URIs** – All relevant app resource must have an identification
  - **Resource state** and not sessions – If resource is modified, next client will see the changes. No session handling!
  - **Uniform interface** – No need for UDDI/WSDL, etc.

?

REST - Representational State Transfer

RESTful in practice

Every resource is identified with an URI:

www.parts-store.org/parts

or

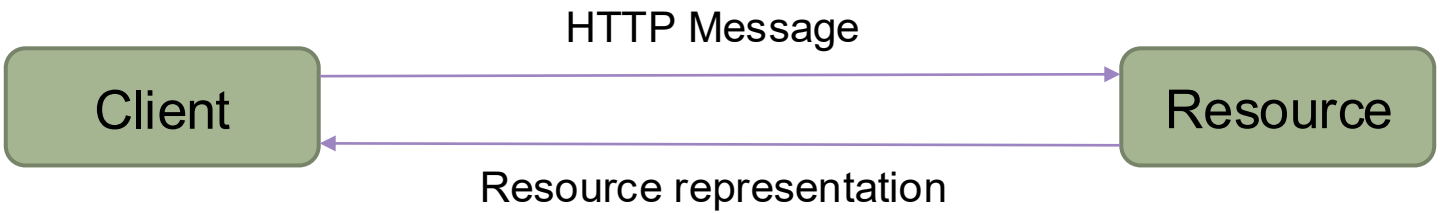
www.parts-store.org/parts/099

and never

www.parts-store.org/parts/getpart?id=099

Nouns (resources), not verbs!

Remember:



REST - Representational State Transfer

RESTful in practice

- **Identifying resources:**
  - It is a hard task…
  - Ideally, any concept within the system that has a **representation** should be exposed as a resource [2]

It is like learning Object-Oriented Language after Procedural Language

(SOAP Web Services have only one URI: the endpoint)

REST - Representational State Transfer

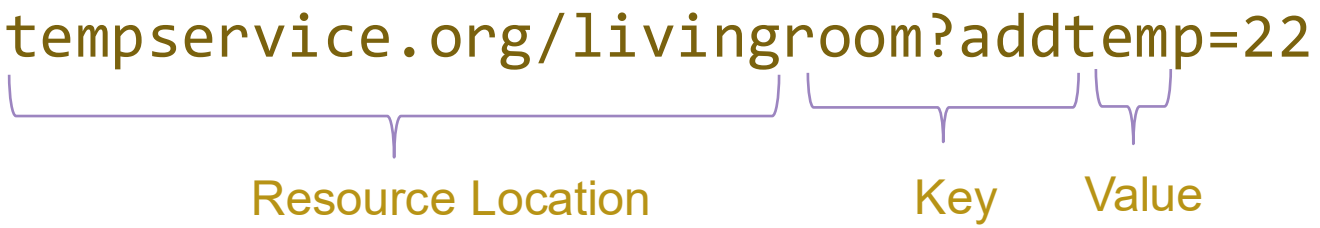
- RESTful in practice
  - Identifying resources:
    - Can we use a single URI?



REST - Representational State Transfer

RESTful in practice

- Identifying resources:
  - Can we use a single URI?
    - Yes, we can, since parameters and operations are encoded in the **query strings**…



This is **not** a good practice. It does not rely on **PUT** HTTP method (update resource)

REST - Representational State Transfer

RESTful in practice

Operations:

REST follows the CRUD model (DBs)

Creating, Retrieving, Updating, and Deleting a resource

HTTP Verbs:

POST - creates a new resource.
GET - retrieves one or many resources.
PUT - updates an existing resource.
DELETE - deletes an existing resource.

90% of web traffic is GET ☺

REST - Representational State Transfer

RESTful in practice

- **Application state:**
  - Server **can not remember** state of each client (**Web Scale**!)
    - Clients have the **responsibility** to remember the state
    - Clients usually send the state in the request in order to provide sufficient application context to the server side

Example:

Client as received an authorization token…
Client includes the token in all subsequent requests
→ Usually in the (Authorization) HTTP request header

REST - Representational State Transfer

RESTful in practice

Errors:

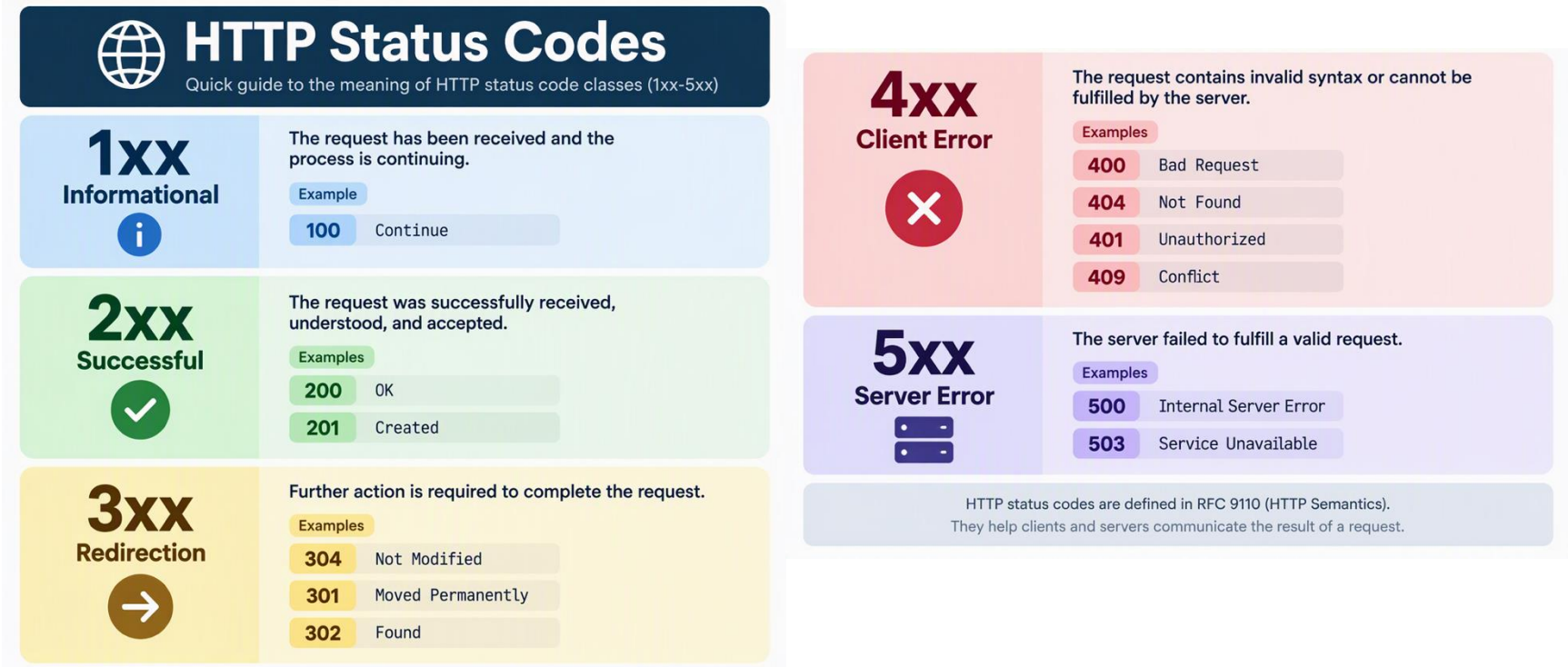
Some people returns **Status Code="200 OK"**, and include the App error in the response body…

This is not a good practice as it forces clients to build **specialized**, non-interoperable software to handle them…

REST - Representational State Transfer

RESTful in practice

Errors (HTTP response code classes) :



REST - Representational State Transfer

RESTful in practice

RESTful Web Service and HTTP Methods

- One URI per resource
  - Multiple operations per URI
- Get all resources / single resource by ID

  GET http://myservice.com/api/books

  GET http://myservice.com/api/books/3
- Add a new resource

  POST http://myservice.com/api/books

REST - Representational State Transfer

RESTful in practice

RESTful Web Service and HTTP Methods

- Modify (update) a resource

  PUT http://myservice.com/api/books/3

- Delete (remove) a resource

  DELETE http://myservice.com/api/books/3

- Update resource fields (partial update)

  PATCH http://myservice.com/api/books/3

Universidade Leiria • Oeste
Escola de Tecnologia e Gestão

Creating RESTful Web Services
ASP.NET Core Web API

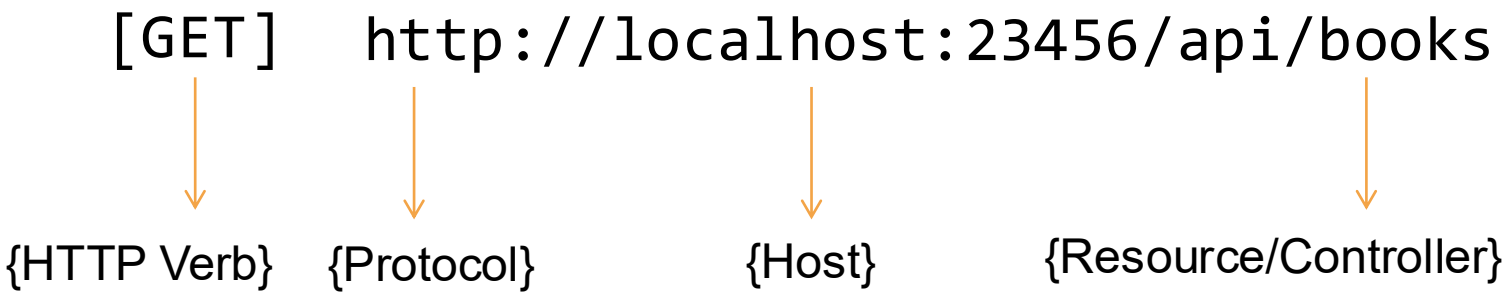




REST - Representational State Transfer

RESTful in practice

Identifying resources:



REST - Representational State Transfer

RESTful in practice

Creating RESTful Web Services…

1. **Using WCF – needs some configuration in order to tune it for REST (default is SOAP!)**
   - Used for special aims (messages queues, duplex communication, TCP, Named pipes, etc.)
2. **Using ASP.NET Core Web API**
   - For resource-oriented services ☺
   - For clients like desktops, smart phones, tablets, etc.
   - Full HTTP compliant (URI, headers, MIME Types, etc.)

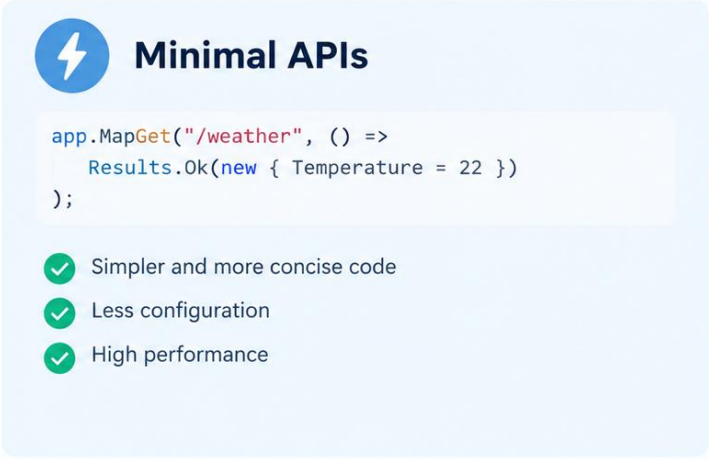
REST - Representational State Transfer

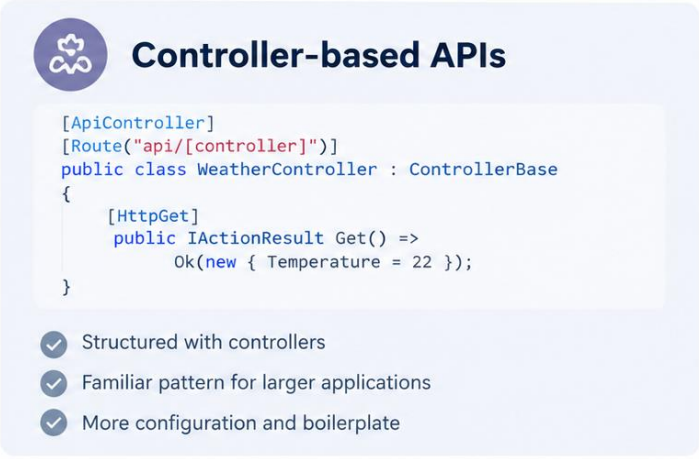
RESTful in practice

Creating RESTful Web Services…

ASP.NET Core provides two approaches for building HTTP APIs:

- **Minimal APIs**
- **Controller-based APIs**





REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Core Web API

New project | **ASP .NET Core Web API**

ProductsAPI (name)

```
dotnet new webapi -n ProductsAPI --controllers
```

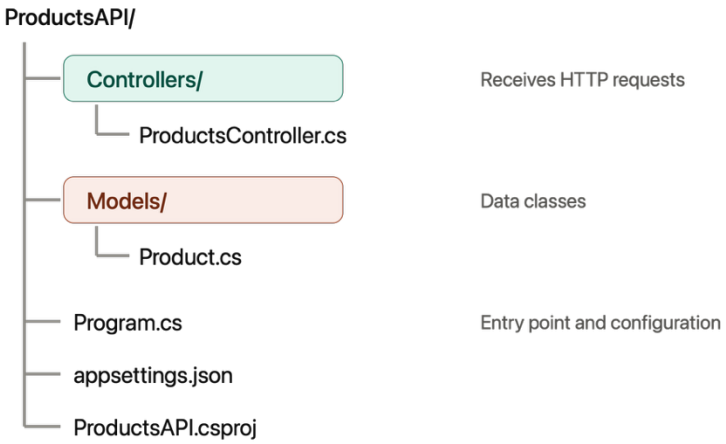
A project skeleton is created…

REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Core Web API

*Now it is time to define which data will be exchanged (**model**) and how to handle http requests (**controller**):*



In the **Models** folder, create class **Product**

This defines the **Product** class with serialization capabilities, to be possible to exchange Product records

REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Core Web API

Product class implementation:

```
namespace ProductsAPI.Models
{
    public class Product
    {
        public int Id { get; set; }
        public string? Name { get; set; }
        public string? Category { get; set; }
        public decimal Price { get; set; }
    }
}
```

Public member variables are also an option

Only **public** members become serializable!

REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Core Web API

*Now it is time to define how, and which HTTP requests are handled (**controller**):*

Add a Controller class

Name it as **Products**Controller and do not touch the word Controller

REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Core Web API

```
namespace ProductsAPI.Controllers
{
    [ApiController]
    //[Route(”api/[controller]")]
    public class ProductsController : ControllerBase
    {
...
    }
}
```

HTTP requests will be redirected to this class

REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Core Web API

```
public class ProductsController : ControllerBase
{
        private  Product[] products = new Product[]
        {
              new Product { Id = 1, Name = "Tomato Soup", Category = "Groceries", Price = 1 },
              new Product { Id = 2, Name = "Yo-yo", Category = "Toys", Price = 3.75M }
        };
        [HttpGet(“api/products”)]
        public Product[] GetAllProducts()
        {
              return products;
        }
        [HttpGet(“api/products/{id:int}”)]
        public IActionResult GetProduct(int id)
        {
              foreach (Product p in products)
              {
                    if (p.Id == id) return Ok(p);
              }
              return NotFound();
        }
...
```

REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Core Web API

```
public IActionResult GetProduct(int id)
{
            foreach (Product p in products)
            {
                        if (p.Id == id) return Ok(p);
            }
            return NotFound();
 }
```



But why not:

```
public Product GetProduct(int id)
{
            foreach (Product p in products)
            {
                if (p.Id == id) return (p);
            }
            return new Product(0,"","",0);
}
```

REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Core Web API

```
public IActionResult GetProduct(int id)
{
            foreach (Product p in products)
            {
                if (p.Id == id) return Ok(p);
            }
            return NotFound();
 }
```

But, why not:

```
public Product GetProduct(int id)
{
            foreach (Product p in products)
            {
                if (p.Id == id) return (p);
            }
            return new Product(0,"","",0);
}
```

It works but… do not reinvent the wheel!

REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Core Web API

Debug, Start debugging…

After running the project…

Append **/api/products** (or **/api/products/1)** to the browser's address bar…

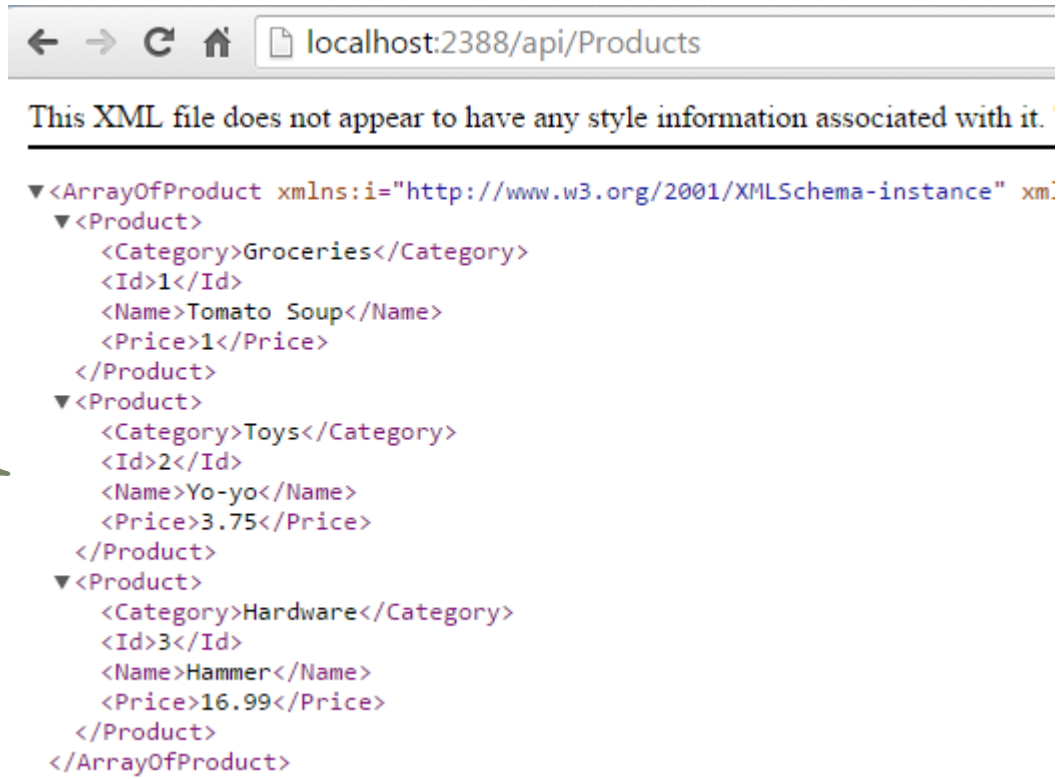
You can now retrieve all products from the REST web service.

REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Core Web API

GET ▸ using browser

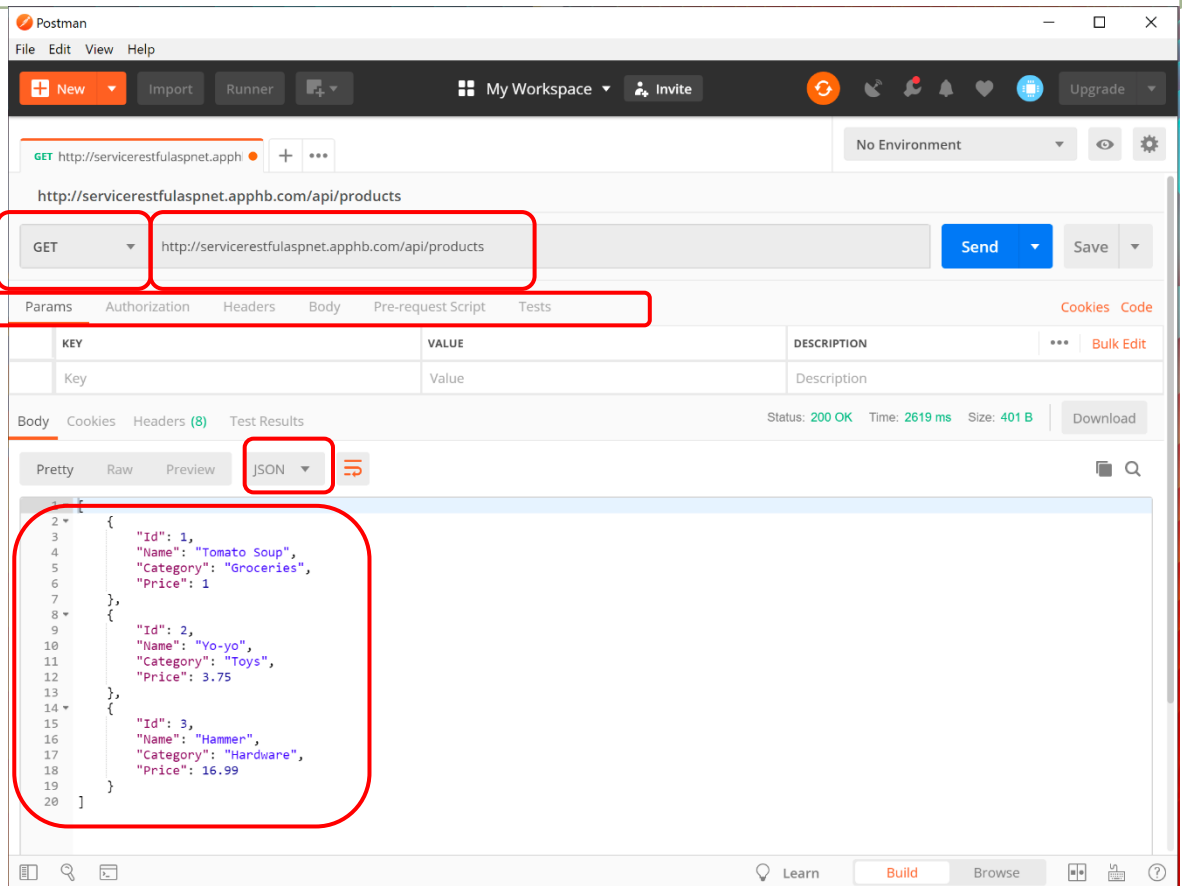


REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Core Web API

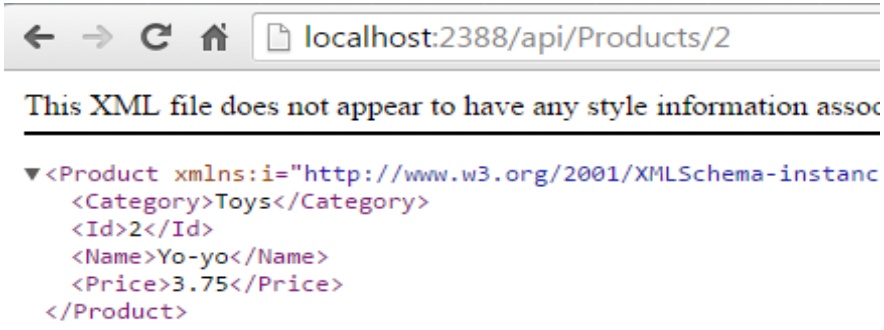
GET ▸ using Postman



REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Core Web API



REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Core Web API

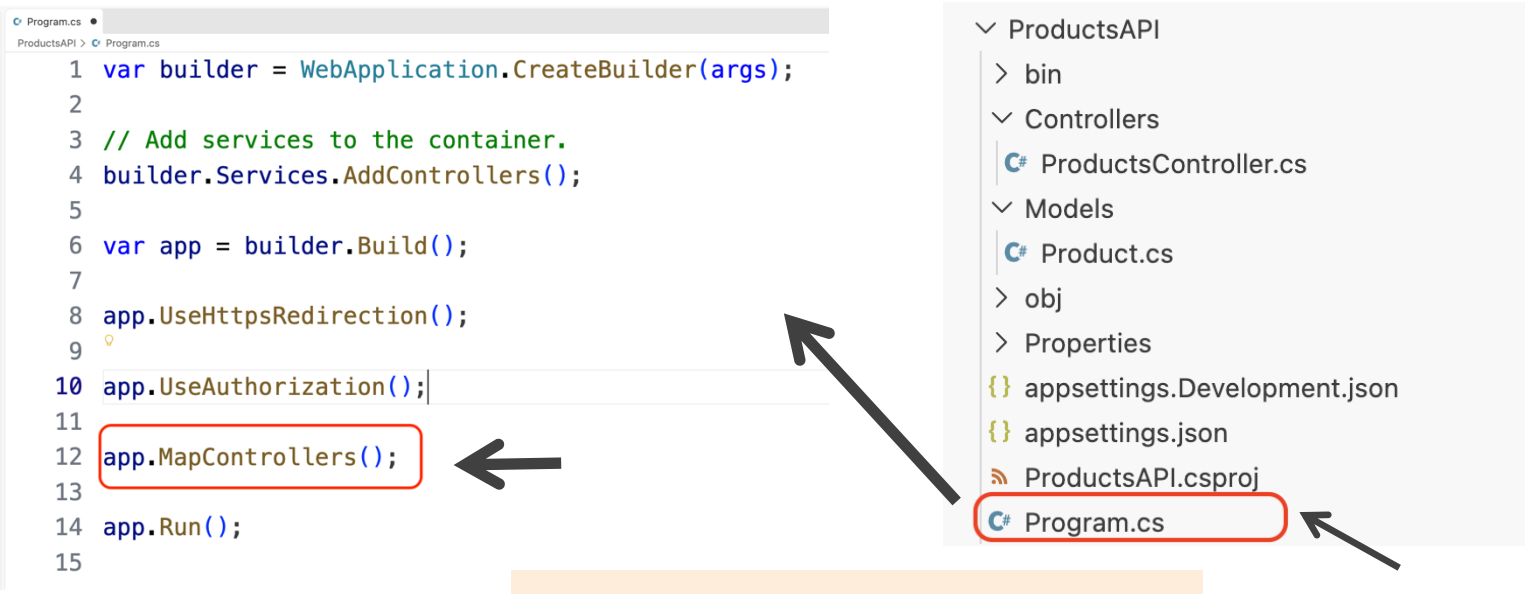
But how HTTP requests reach the right controller?

HttpRoutes!

Program.cs

ProductsAPI > Program.cs

```
1  var builder = WebApplication.CreateBuilder(args);
2
3  // Add services to the container.
4  builder.Services.AddControllers();
5
6  var app = builder.Build();
7
8  app.UseHttpsRedirection();
9
10 app.UseAuthorization();
11
12 app.MapControllers();
13
14 app.Run();
15
```



(You can add more models and controllers)

REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Core Web API

All CRUD methods:

```
[HttpGet("api/products")]
public IActionResult GetAllProducts()
{
      return Ok(products);
}
[HttpGet("api/products/{id:int}")]
public IActionResult GetProduct(int id)
{
       … return Ok(product);

       return NotFound();
 }
```

/api/products

/api/products/2

REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Core Web API

```
[HttpPost("api/products")]
public IActionResult PostProduct(Product p)
{
        //Save product and…
        return Ok(p);

              //Any error or exception…
              return NotFound();
}
```

POST /api/products

REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Core Web API

```
[HttpPut("api/products/{id:int}")]
public IActionResult PutProduct(int id, Product p)
{
        //Update product for p.id and…
        return Ok(p);

              //Any error or exception…
              return NotFound();
}
```

PUT /api/products/2

REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Core Web API

```
[HttpDelete("api/products/{id:int}")]
public IActionResult DeleteProduct(int id)
{
        //Delete record and
              return Ok();

        //If error or exception
              return NotFound();
}
```

DELETE /api/products/3

REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Core Web API

How to try Post, PUT and Delete operations?

Use Javascript/Ajax, download **Fiddler** free tool, browser extensions, cURL, etc.

Other tools: **Postman**, Bruno, Insomnia, cURL, etc.





REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Core Web API

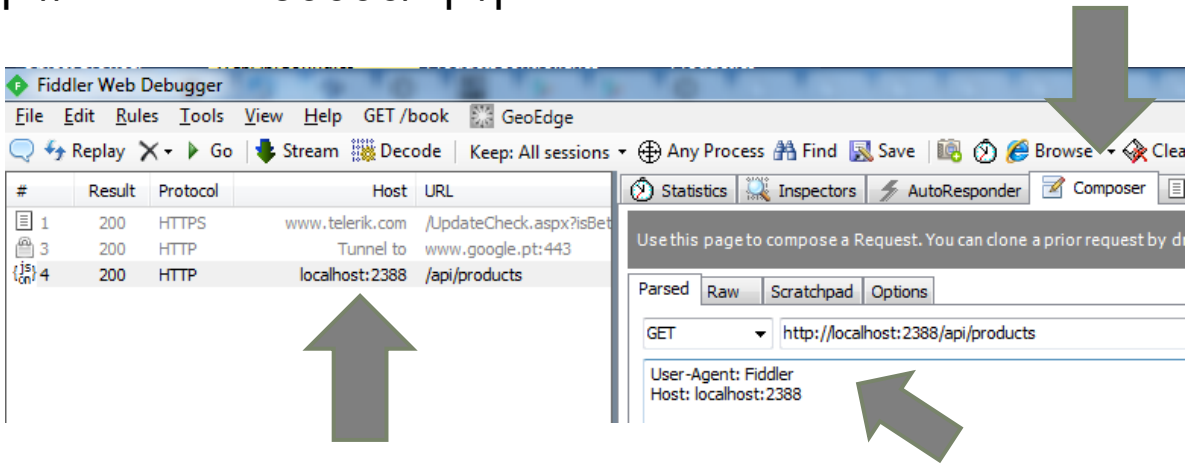
How to try Post, PUT and Delete operations?

Fiddle:

Composer tab,

GET http://localhost:30606/api/products

GET



REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Core Web API

How to try Post, PUT and Delete operations?

Composer tab,

GET http://localhost:2388/api/products/1

GET

REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Core Web API

How to try Post, PUT and Delete operations?

Composer tab,

POST http://localhost:30606/api/products

POST

Request header: Content-Type: **application/json**; charset=utf-8

Request body: [{"Id":"6","Name":"Tomato salad","Category":"Groceries","Price":"3"}]

Use a breakpoint on service::post to see request

REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Core Web API

How to try Post, PUT and Delete operations?

Composer tab,

PUT

PUT http://localhost:30606/api/products/1

Request header: Content-Type: **application/json**; charset=utf-8

Request body: [{"Id":"1","Name":"Tomato Soup","Category":"Groceries","Price":"1"}]

Use a breakpoint on service::put to see request

REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Core Web API

How to try Post, PUT and Delete operations?

Composer tab,

DELETE http://localhost:2388/api/products/5

DELETE

Use a breakpoint on service::delete to see request

REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Core Web API

Additional configurations

- Attribute routing
- URI template with several parameters
- Http methods

REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Core Web API

More on attribute routing

`/customers/1/orders`

- This type of URI is difficult to create using convention-based routing
- Although it can be done, the results don't scale well if you have many controllers or resource types

```
[HttpGet("customers/{customerId:int}/orders")]
public IEnumerable<Order> GetOrdersByCustomer(int customerId) { ... }
```

REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Web API

More on attribute routing?

```
[HttpGet("customers/{customerId:int}/orders")]
public IEnumerable<Order> GetOrdersByCustomer(int customerId) {
... }
```

- The string "**customers/{customerId}/orders**" is the URI template for the route
- Web API tries to match the request URI to the template
- "customers" and "orders" are literal segments, and "{customerId}" is a variable parameter

REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Web API

Adding routing attributes

```
[HttpGet("customers/{customerId:int}/orders")]
public IEnumerable<Order> GetOrdersByCustomer(string customerId)
{ ... }
```

```
http://localhost/customers/1/orders
http://localhost/customers/bob/orders
http://localhost/customers/1234-5678/orders
```

REST - Representational State Transfer

- RESTful in practice
  - Creating a RESTful Web Service using ASP.NET Web API
    - **A URI template can have several parameters**

```
[HttpGet("customers/{customerId:int}/orders/{orderId:int}")]
public Order GetOrderByCustomer(int customerId, int orderId)
{ ... }
```

REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Web API

Http methods

- Web API selects actions based on the HTTP method of the request (GET, POST, etc.)
- API looks for a **case-insensitive match** with the start of the **controller method name**
- For example, a controller method named *PutCustomers* matches an *HTTP PUT* request

REST - Representational State Transfer

- RESTful in practice
  - Creating a RESTful Web Service using ASP.NET Web API
    - **Http methods**

- Override conventions by decorating the method with any the following attributes:

[HttpDelete] **[HttpGet]** **[HttpHead]** **[HttpOptions]**	**[HttpPatch]** **[HttpPost]** **[HttpPut]**

```
[HttpPost("api/books")]
public IActionResult CreateBook(Book book) { ... }
```

REST - Representational State Transfer

RESTful in practice

Creating a RESTful Web Service using ASP.NET Web API

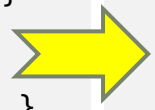
Route prefixes

Often, the routes in a controller all start with the same prefix (use Route)

```
[ApiController]
public class BooksController : ControllerBase
{
    [HttpGet(”api/books")]
    public IEnumerable<Book> GetBooks() { ... }

    [HttpGet(”api/books/{id:int}")]
    public IActionResult GetBook(int id) { ... }

    [HttpPost(”api/books")]
    public IActionResult CreateBook(Book book) {
... }
}
```



```
[ApiController]
[Route (”api/[controller]")]
public class BooksController : ControllerBase
{
    // GET api/books
    [HttpGet("")]
    public IEnumerable<Book> Get() { ... }

    // GET api/books/5
    [HttpGet("{id:int}")]
    public IActionResult Get(int id) { ... }

    // POST api/books
    [HttpPost("")]
    public IActionResult Post(Book book)
    { ... }
}
```

REST - Representational State Transfer

RESTful in practice

- The Kestrel (embedded) Web Server*
  - Configured by default in ASP.NET Core project templates

```
var builder =
WebApplication.CreateBuilder(args);

var app = builder.Build();

app.MapGet("/", () => "Hello World!");

app.Run();
```

Calls UseKestrel internally...

*https://learn.microsoft.com/en-us/aspnet/core/fundamentals/servers/kestrel?view=aspnetcore-10.0

REST - Representational State Transfer

RESTful in practice

Deployment the REST Web Service using VS Code

1. Publish service using VS Code

```
dotnet publish -c Release -o publish
```

Execute the service:

```
dotnet publish/ProductsAPI.dll
```

Test the endpoint at
`http://localhost:5000/api/products`

For self-exe(s):
dotnet publish -c Release -r osx-arm64 --self-contained true
dotnet publish -c Release -r linux-x64 --self-contained true
dotnet publish -c Release -r win-x64 --self-contained true

REST - Representational State Transfer

RESTful in practice

Deployment of the REST Web Service using Apache (macOSX)

dotnet publish -c Release -o publish

```
cd publish; dotnet ProductsAPI.dll (shows port)
```

```
vim /opt/homebrew/etc/httpd/httpd.conf
```

```
Uncomment mod_proxy.so and mod_proxy_http.so
```

```
At the end:
ProxyPreserveHost On
ProxyPass / http://localhost:5000/
ProxyPassReverse / http://localhost:5000/
```

brew services restart httpd
http://localhost:8080/api/products

Well…Till now…

Request-response model of interaction…

1. If the server fails…
2. Server is working again…
3. Server returns a response to the client, but client fails…
4. No contention. If a million requests take place…
…



Well…Till now…

Request-response model of interaction…

1. If the server fails…
2. Server is working again…
3. Server returns a response to the client, but client fails…
4. No contention. If a million requests take place…
…

Is there any alternative?

Messaging (message-oriented middleware)

Rabbit MQ, BeanTalkd, ActiveMQ, IBM MQ Series, Tuxedo, …

1. If the server fails, **queue persists messages, even shutdown**
2. Server is working again; **it receives pending messages…**
3. Server returns a response to the client, but client fails. **Response is persisted and delivered when possible…**
4. No contention. If a million requests take place. **Now we can decide how many requests are handled by the server…**

References

[1] Paul Prescod, “Roots of the REST/SOAP Debate”, Proceedings of Extreme Markup Languages®, 2002

[2] Erik Wilde and Cesare Pautasso, “REST: From Research to Practice”, Springer, 2011